Experiment -1

1. A class consists of 40 students who have secured different grades in an examination. The grades are
classified as F, E, D, C, B, A, and S, where F represents the lowest grade and S represents the highest grade.
The number of students who secured each grade is given below.

Grade	Number of Students
F	3
E	4
D	5
C	8
B	9
A	7
S	4
Total	40

Develop a Python program to perform the following tasks:
1. Store the grades and corresponding number of students in suitable data structures.
2. Draw a pie chart to represent the percentage distribution of students across the seven grades.
3. Display the grade labels and percentage values on the pie chart.
4. Draw a bar chart to compare the number of students who secured each grade.
5. Label the X-axis as Grades and the Y-axis as Number of Students.
6. Display the exact number of students above each bar.
7. Add suitable chart titles, legends, and axis labels.
8. Identify the grade secured by the maximum and minimum number of students.
9. Display the percentage of students who secured grades A and S combined.

Aim: To write a Python program using Matplotlib to represent student grade distribution using pie and bar charts, identify the maximum and minimum grades, and calculate the combined percentage of students who secured grades A and S.

PYTHON CODE :

```
import matplotlib.pyplot as plt

# 1. Store grades and number of students
grades = ['F', 'E', 'D', 'C', 'B', 'A', 'S']
students = [3, 4, 5, 8, 9, 7, 4]
```

```
total_students = sum(students)

# Calculate percentage distribution
percentages = [(n / total_students) * 100 for n in students]

# 2 & 3. Draw pie chart with labels and percentages
plt.figure(figsize=(8, 6))

plt.pie(
    students,
    labels=grades,
    autopct='%1.1f%%',
    startangle=90,
    shadow=False
)

plt.title('Percentage Distribution of Students by Grade')
plt.legend(grades, title='Grades', loc='best')
plt.tight_layout()
plt.show()

# 4, 5 & 6. Draw bar chart with exact values
plt.figure(figsize=(8, 6))

bars = plt.bar(grades, students, color='skyblue', edgecolor='black')

plt.xlabel('Grades')
plt.ylabel('Number of Students')
plt.title('Number of Students in Each Grade')

# Display exact number above each bar
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 0.1,
        str(int(height)),
        ha='center',
        va='bottom'
```

```
    )

plt.legend(['Number of Students'])
plt.ylim(0, max(students) + 2)
plt.tight_layout()
plt.show()

# 8. Identify maximum and minimum grades
max_students = max(students)
min_students = min(students)

max_grades = [
    grades[i] for i in range(len(grades))
    if students[i] == max_students
]

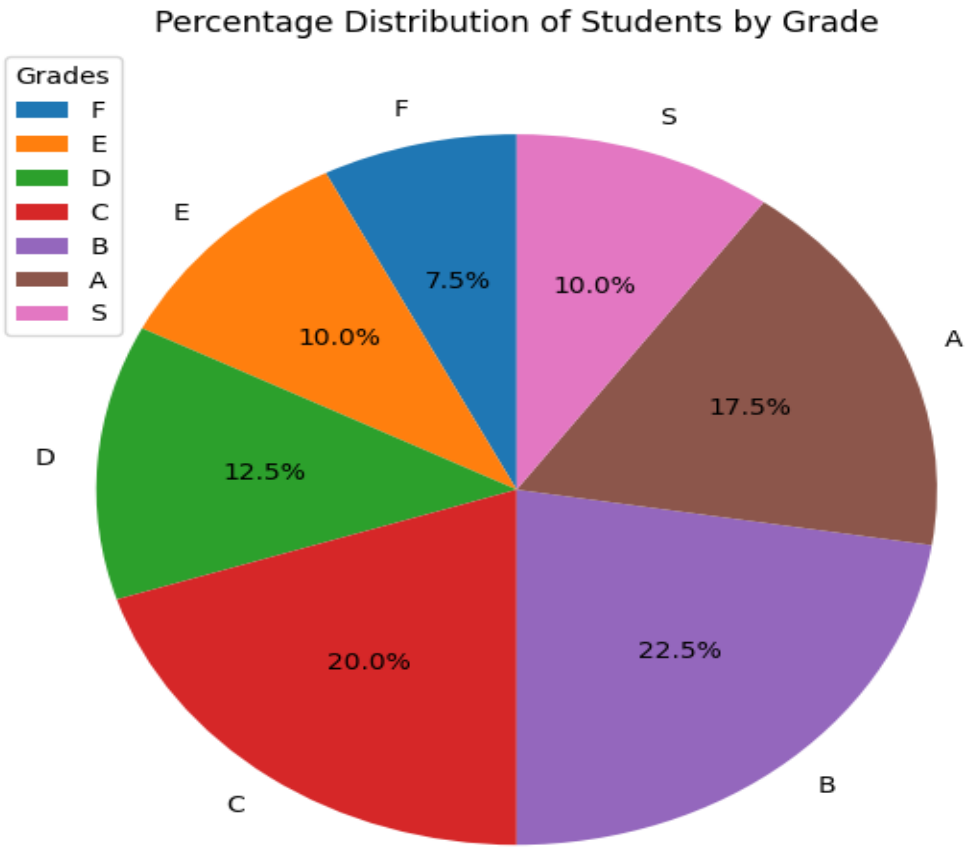
min_grades = [
    grades[i] for i in range(len(grades))
    if students[i] == min_students
]

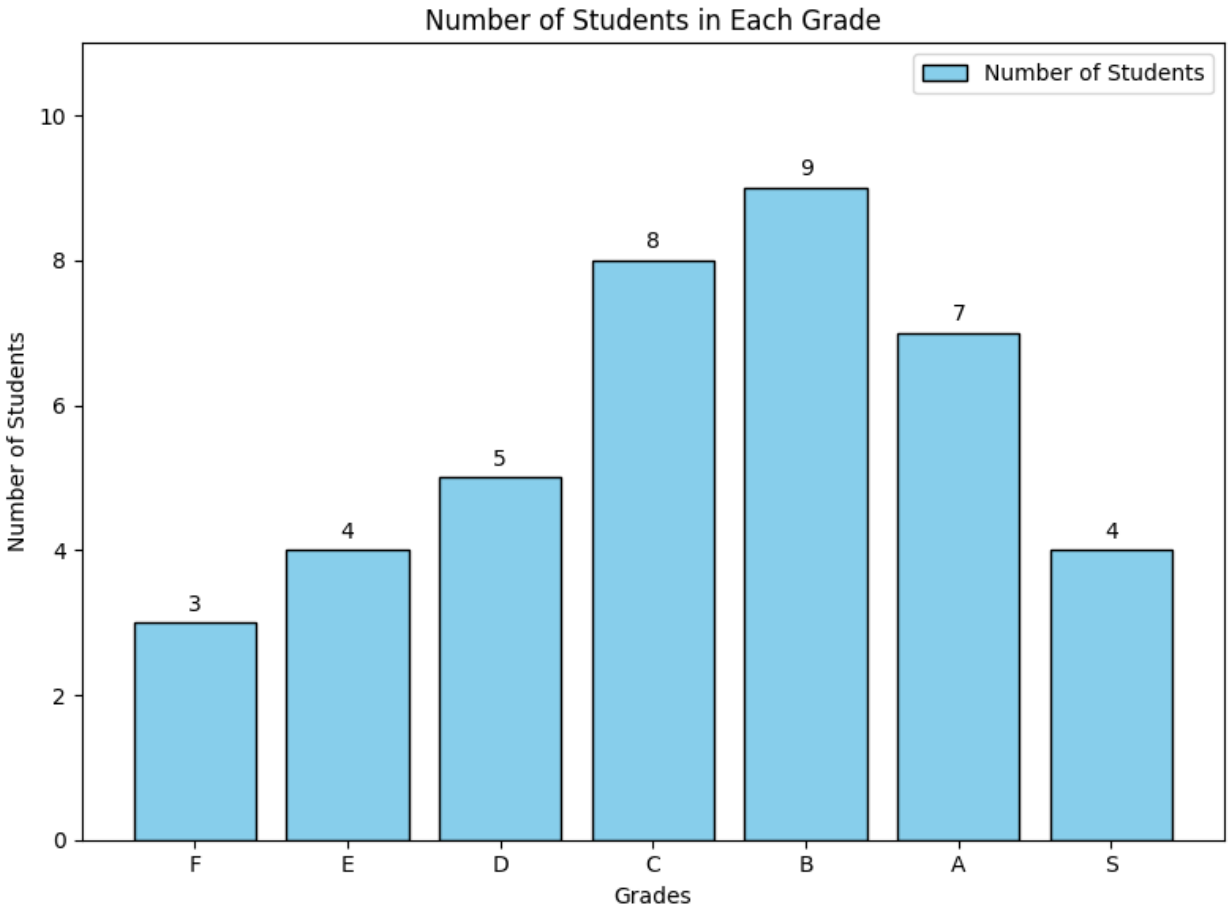
print("Total number of students:", total_students)
print("Grade(s) with maximum students:",
      ', '.join(max_grades), "-", max_students, "students")
print("Grade(s) with minimum students:",
      ', '.join(min_grades), "-", min_students, "students")
a_s_students = students[grades.index('A')] + students[grades.index('S')]
a_s_percentage = (a_s_students / total_students) * 100

print("Students who secured A and S combined:", a_s_students)
print("Percentage of students who secured A and S combined:",
      f"{a_s_percentage:.2f}%")
```

OUTPUT:

```
Total number of students: 40
Grade(s) with maximum students: B - 9 students
Grade(s) with minimum students: F - 3 students
Students who secured A and S combined: 11
Percentage of students who secured A and S combined: 27.50%
```


Percentage Distribution of Students by Grade
Grades
F
E
D
C
B
A
S
F
S
E
A
D
C
B
7.5%
10.0%
10.0%
17.5%
12.5%
20.0%
22.5%



Result: The program successfully displayed the pie chart and bar chart, identified grade B as the maximum (9 students) and grade F as the minimum (3 students), and calculated the combined percentage of grades A and S as 27.50%.

EXPERIMENT-2

2) Semantic Network Using NetworkX
Aim: To represent concepts and labeled relationships as a directed semantic network.
Software Required: Python 3.x and NetworkX (install with: pip install networkx)
Procedure / Algorithm
1. Create a directed graph.
2. Add concepts as nodes implicitly through edges.
3. Add labeled relationships.
4. Display each relationship.
Program
import networkx as nx
G = nx.DiGraph()
G.add_edge("Dog", "Mammal", relation="is-a")
G.add_edge("Mammal", "Animal", relation="is-a")
G.add_edge("Dog", "Tommy", relation="has-instance")
print("Semantic Network Relationships:")
for source, target, data in G.edges(data=True):
print(source, "--", data["relation"], "-->", target)
Sample Input
Concepts and relations are predefined in the program.
Expected Output
Semantic Network Relationships:
Dog -- is-a --> Mammal
Dog -- has-instance --> Tommy
Mammal -- is-a --> Animal
Result: The semantic network represented concepts and their relationships.

Aim: To write a Python program using NetworkX to represent concepts and their labeled relationships as a directed semantic network.

PYTHON CODE:

```
import networkx as nx
import matplotlib.pyplot as plt

# 1. Create a directed graph
G = nx.DiGraph()
```

```
# 2 & 3. Add concepts and labeled relationships
G.add_edge("Dog", "Mammal", relation="is-a")
G.add_edge("Mammal", "Animal", relation="is-a")
G.add_edge("Dog", "Tommy", relation="has-instance")

# 4. Display each relationship
print("Semantic Network Relationships:")
for source, target, data in G.edges(data=True):
    print(source, "--", data["relation"], "-->", target)

# 5. Draw the semantic network
pos = nx.spring_layout(G, seed=42)

plt.figure(figsize=(9, 6))

nx.draw_networkx_nodes(
    G, pos,
    node_color="lightblue",
    node_size=2500,
    edgecolors="black"
)

nx.draw_networkx_labels(
    G, pos,
    font_size=12,
    font_weight="bold"
)

nx.draw_networkx_edges(
    G, pos,
    arrows=True,
    arrowstyle="-|>",
    arrowsize=20,
    width=2,
    connectionstyle="arc3,rad=0.05"
)

# Display relationship labels on edges
edge_labels = nx.get_edge_attributes(G, "relation")
nx.draw_networkx_edge_labels(
```

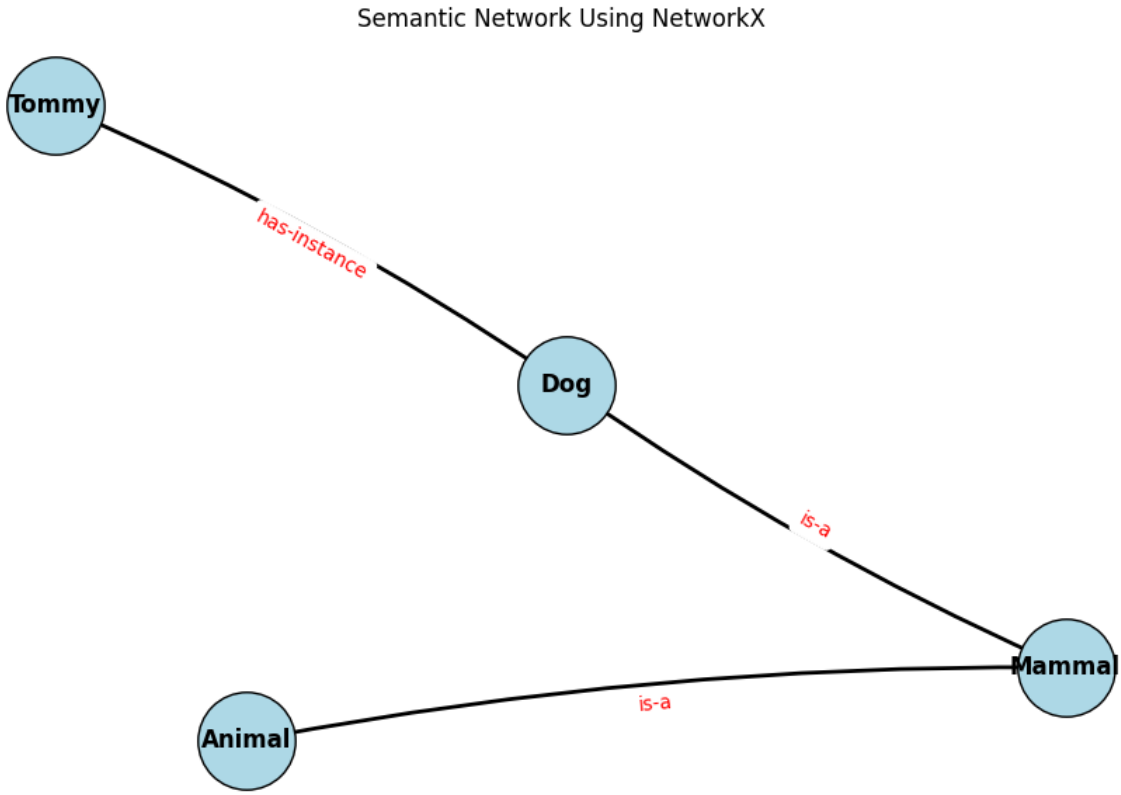
```
    G, pos,
    edge_labels=edge_labels,
    font_size=10,
    font_color="red",
    label_pos=0.5
)

plt.title("Semantic Network Using NetworkX")
plt.axis("off")
plt.tight_layout()
plt.show()

# Result
print("\nResult: The semantic network represented concepts and their relationships.")
```

OUTPUT:

Semantic Network Relationships:
Dog -- is-a --> Mammal
Dog -- has-instance --> Tommy
Mammal -- is-a --> Animal



Result:

The semantic network represented concepts and their relationships.

EXPERIMENT -3

3) Frame-Based Knowledge Representation
Aim: To represent an object as a frame containing slots and values.
Software Required: Python 3.x

Procedure / Algorithm
1. Create a dictionary to represent a frame.
2. Use keys as slots and values as slot values.
3. Display the frame.
Program
vehicle = {
"class": "Vehicle",
"brand": "Toyota",
"fuel_type": "Petrol",
"wheels": 4,

"color": "White"
}
print("Vehicle Frame")
for slot, value in vehicle.items():
print(slot, ":", value)
Sample Input
Frame values are predefined in the program.
Expected Output
Vehicle Frame
class : Vehicle
brand : Toyota
fuel_type : Petrol
wheels : 4
color : White
Result: The vehicle was represented using slots and their corresponding values.

Aim: To write a Python program to represent an object using a frame containing slots and their corresponding values.

Python Code:

```
import matplotlib.pyplot as plt

# 1. Create a frame using a dictionary
vehicle = {
    "class": "Vehicle",
    "brand": "Toyota",
    "fuel_type": "Petrol",
    "wheels": 4,
    "color": "White"
}

# 2. Display the frame
print("Vehicle Frame:")
for slot, value in vehicle.items():
    print(slot, ":", value)

# 3. Draw the frame diagram
```

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.axis("off")

# Frame title
ax.set_title("Frame-Based Knowledge Representation",
             fontsize=14, fontweight="bold", pad=20)

# Create table for slots and values
table_data = [["Slot", "Value"]]
for slot, value in vehicle.items():
    table_data.append([slot, str(value)])

table = ax.table(
    cellText=table_data,
    cellLoc="center",
    loc="center",
    colWidths=[0.4, 0.4]
)

# Format the table
table.auto_set_font_size(False)
table.set_fontsize(12)
table.scale(1, 1.8)

# Color the header
for col in range(2):
    table[(0, col)].set_facecolor("lightblue")
    table[(0, col)].set_text_props(weight="bold")

plt.tight_layout()
plt.show()

# 4. Display result
print("\nResult: The vehicle was represented using slots and their corresponding values.")
```

OUTPUT:

```
Vehicle Frame:
class : Vehicle
brand : Toyota
fuel_type : Petrol
```

wheels : 4
color : White

Frame-Based Knowledge Representation

Slot	Value
class	Vehicle
brand	Toyota
fuel_type	Petrol
wheels	4
color	White

Result:

The program successfully represented a vehicle as a frame using a dictionary, storing and displaying its class, brand, fuel type, number of wheels, and color.